V-OS

Un sistema operativo gráfico para el Amstrad CPC 6128

Manual

Hecho a partir de docs/manual/es/*.md y sdk/README.es.md con tools/mkdocs.py (make docs).

Índice

V-OS es un sistema operativo gráfico para el Amstrad CPC 6128, escrito desde cero en ensamblador Z80. Tiene:

- una barra de menús, ventanas que se pueden arrastrar, diálogos y botones, que se usan con el teclado o con un ratón AMX
- tres vistas para los archivos y los programas (Escritorio, Programas, Commander) y los ajustes (Ajustes)
- seis idiomas como complementos: inglés, griego, francés, español, alemán e italiano
- programas que funcionan uno al lado del otro y continúan donde se quedaron, un accesorio de escritorio y programas en segundo plano
- Studio, un editor de bloques al estilo de Scratch que funciona en el propio CPC
- una API con 42 llamadas y un SDK para programas en ensamblador

Este manual también está en griego: 00-index.md, en inglés: 00-index.md, en francés: 00-index.md, en alemán: 00-index.md y en italiano: 00-index.md.

- 1. Primeros pasos
- 2. Las vistas
- 3. Programas
- 4. Idiomas
- 5. Studio: programas con bloques
- 6. Bloques en el PC: el editor web y `vbc.py`
- 7. El SDK de V-OS: programas en ensamblador
- 8. Desarrollo de V-OS

1. Primeros pasos

Qué se necesita

- Un Amstrad CPC 6128 o un emulador. V-OS se prueba en Caprice32.
- El disquete de V-OS en la unidad A:, es decir, el `build/vos.dsk` que genera `make`.
- De forma opcional:
  - un ratón AMX
  - una segunda unidad (B:)
  - una ampliación de memoria (p. ej. 512K), para más programas a la vez

El arranque

Desde el BASIC:

```
RUN"VOS
```

V-OS carga el núcleo, desactiva las ROM y toma el control de toda la máquina. Detecta por sí solo la ampliación de memoria.

- **Primera vista:** el Escritorio. Se puede cambiar en Ajustes, en la línea VISTA INICIAL.
- **Idioma:** inglés. Si hay un `SETUP.CFG` en el disquete, arranca con el idioma y los ajustes guardados en él.
- **Salir:** ARCHIVO > SALIR AL BASIC (CONTROL+Q en las vistas). Pide confirmación y reinicia el CPC.

La pantalla

V-OS usa toda la pantalla (overscan): 384×272 píxeles en modo 1 (4 colores) o 192×272 en modo 0 (16 colores).

- **Barra de menús** (arriba): los menús de la vista o del programa, normalmente ARCHIVO o el menú del programa, VER y AYUDA. A la derecha está el idioma del teclado (EN, EΛ, ...).
- **Línea inferior:** en cada vista, las teclas que funcionan en ella.

Teclas que funcionan en todas partes

Tecla	Qué hace
F1	Escritorio
F2	Programas
F3	Commander
F4	Ajustes
F5	abre y cierra el accesorio de escritorio (ACCESORIO)
F6	los programas en marcha, para elegir uno
ESC	abre el primer menú. En los menús, ←/→ cambian de menú, ↑/↓ eligen una opción, RETURN la ejecuta, ESC cierra
CONTROL+letra	los atajos de los menús (p. ej. ^Q, ^L)

CONTROL+L	cambio de idioma (VER > CAMBIAR DE IDIOMA)
CONTROL+K	el diálogo con las teclas (AYUDA > TECLAS)
CONTROL+SHIFT	cambia el teclado entre el inglés y el del idioma del paquete
TAB, SHIFT+TAB	mueven el foco entre los botones. RETURN, ENTER o SPACE pulsan el botón que tiene el foco

F1–F6 son las teclas f1–f6 del teclado numérico del CPC.

El puntero y el ratón

- **Con un ratón AMX:** el puntero se mueve con el ratón, y su botón hace clic. El ratón se activa y se configura en Ajustes (RATÓN AMX, LECTURA AMX).
- **Sin ratón:** el puntero se mueve con las flechas, y COPY hace clic. En las vistas que usan las flechas para elegir (Escritorio, Programas, Commander), las flechas eligen archivos y programas en lugar de mover el puntero.

Casi todo funciona de las dos maneras: un clic en un archivo o programa lo selecciona, y un segundo clic lo abre.

Ventanas y diálogos

- **Arrastrar:** todas las ventanas se pueden arrastrar por su barra de título: los diálogos, la ventana ARCHIVOS del Escritorio, el accesorio de escritorio y las ventanas de los programas de bloques. Mientras se mantiene pulsado el botón se ve un contorno discontinuo, y la ventana se vuelve a dibujar donde se suelta.
- **Diálogos:** esperan una respuesta. TAB y ←/→ cambian de botón, RETURN lo pulsa, y ESC elige el último (normalmente NO o CANCELAR).
- **Introducción de texto** (p. ej. al renombrar): se escribe al final, DEL borra, RETURN acepta y ESC cancela. También se puede escribir en el idioma del paquete.

Disquetes y archivos

- **Formato:** V-OS lee y escribe disquetes AMSDOS (formato DATA o SYSTEM) en A: y B:. El BASIC también puede leer sus archivos.
- **Nombres:** 8 caracteres más 3 para el tipo (`NOMBRE.TIP`).

Tipos que se encontrará:

Tipo	Qué es
`.BIN`	un programa de V-OS (o un archivo binario)
`.VBN`	un programa de V-OS hecho con bloques
`.VBS`	un programa de Studio (su código fuente)
`.OVL`, `.MSG`, `.LNG`	partes del propio V-OS: vistas, textos, idiomas
`.SYS`	archivos del sistema (p. ej. `VBRUN.SYS`, el reproductor de Studio)
`SETUP.CFG`	los ajustes

Los ajustes, los idiomas y las vistas se leen de A:, así que el disquete de V-OS debe quedarse ahí.

Mensajes de error

Los errores de disco se muestran en un diálogo: UNIDAD NO LISTA, DISCO PROTEGIDO, NO SE

ENCUENTRA, DISCO LLENO, DIRECTORIO LLENO, EL NOMBRE YA EXISTE, NOMBRE NO VÁLIDO y otros.

Dos más tienen que ver con los programas:

- **NO ES PROGRAMA V-OS:** el archivo no tiene la cabecera de un programa.
- **CIERRE PRIMERO OTRO PROGRAMA:** no hay sitio. Véase el capítulo 3.

2. Las vistas

V-OS tiene cuatro "vistas": maneras de ver los archivos y los programas, y los ajustes. Se cambia de vista con F1–F4 o desde el menú VER. Cada vista recuerda qué archivo estaba seleccionado, y la siguiente selecciona el mismo archivo.

Los menús comunes

Los dos últimos menús son iguales en todas las vistas y en los programas:

VER		**AYUDA**	
ESCRITORIO	F1	TECLAS	^K
PROGRAMAS	F2	ACERCA DE V-OS	
COMMANDER	F3		
AJUSTES	F4		
PROGRAMAS (en marcha)	F6		
CAMBIAR DE IDIOMA	^L		

Escritorio, F1

Modo 1. Una ventana ARCHIVOS muestra los archivos de una unidad como iconos, cuatro por fila. El tipo va escrito dentro del icono y el nombre debajo, y un `.VBN` lleva al lado una marca de "play". A la derecha, dos iconos eligen A: o B:. La barra de título muestra el formato del disquete y el número de archivos, y el espacio libre se muestra abajo.

Tecla	**Qué hace**
flechas	eligen un archivo (↑/↓ una fila)
TAB	la otra unidad
RETURN, ENTER	información del archivo
^C	copiar a la otra unidad
^D	borrar (pregunta antes)
^N	renombrar
^R	releer los discos
ratón	clic en un archivo: lo selecciona, otra vez: información. Clic en una unidad: la abre

- **Menú ARCHIVO:** EJECUTAR (ejecuta el programa seleccionado), INFO DEL ARCHIVO, COPIAR A LA OTRA UNIDAD, BORRAR, RENOMBRAR, RELEER LOS DISCOS, SALIR AL BASIC.
- **La ventana ARCHIVOS** se puede arrastrar por su título y se queda ahí, incluso al cambiar de vista.

Programas, F2

Modo 0, 16 colores. Muestra solo los programas de A: (`.BIN`, `.VBN`, `.BAS`), como baldosas en una cuadrícula de 3×3, con las páginas siguientes debajo. Cada programa que tiene icono lo muestra en su baldosa. La baldosa seleccionada es blanca.

Tecla	**Qué hace**
flechas	eligen (↑/↓ una fila. Más allá del borde, la página siguiente o la anterior)
RETURN, ENTER	ejecuta el programa. Si ya está en marcha, continúa donde estaba

^A	lo carga como accesorio de escritorio (COMO ACCESORIO). Después F5 lo abre
^R	releer el disco
ratón	clic: selecciona, otra vez: ejecuta

Menú ARCHIVO: EJECUTAR, COMO ACCESORIO, RELEER LOS DISCOS, SALIR AL BASIC.

Commander, F3

Modo 1, dos paneles con listas de archivos: A: a la izquierda, B: a la derecha. Cada panel muestra el formato y el número de archivos en su título, y el espacio libre abajo. El panel activo tiene el título oscuro y una selección.

Tecla	Qué hace
↑/↓	eligen (con SHIFT: una página)
TAB, ←/→	el otro panel
RETURN, ENTER	información: tamaño, tipo, dirección de carga, los primeros bytes
^C	copiar al otro panel, de cualquier tamaño. Si hay un archivo con el mismo nombre, pregunta antes
^D	borrar
^N	renombrar
^R	releer los dos discos
ratón	clic: selecciona, otra vez: información

Menú ARCHIVO: EJECUTAR, INFO DEL ARCHIVO, COPIAR AL OTRO PANEL, BORRAR, RENOMBRAR, RELEER LOS DISCOS, FORMATEAR DISCO (DATA), SALIR AL BASIC.

> FORMATEAR borra todos los archivos del disquete del panel activo. Pregunta antes.

Ajustes, F4

Modo 1. Una línea para cada ajuste. Los cambios se aplican al momento. Al guardarlos (RETURN o ^S) se escriben en `A:SETUP.CFG` y valen también en el siguiente arranque.

Ajuste	Valores
VISTA INICIAL	ESCRITORIO, PROGRAMAS, COMMANDER
IDIOMA	ENGLISH, ΕΛΛΗΝΙΚΑ, FRANÇAIS, ESPAÑOL, DEUTSCH, ITALIANO
RETARDO REPETICIÓN	cuánto tiempo se mantiene pulsada una tecla antes de que empiece a repetirse: CORTO, MEDIO, LARGO
VELOC. REPETICIÓN	LENTA, MEDIO, RÁPIDA
RATÓN AMX	SÍ, NO
LECTURA AMX	PULSOS o NIVELES: cómo se lee el ratón. Si el puntero no se mueve bien, pruebe el otro
COLOR DEL FONDO	AZUL, NEGRO, VERDE, ROJO

Tecla	Qué hace
↑/↓	eligen un ajuste
←/→	cambian el valor (también un clic en la línea)
RETURN, ^S	guardar

Al cambiar el IDIOMA se carga al momento el paquete de idioma desde A:. Véase el capítulo 4.

3. Programas

Empezar y terminar

Un programa se inicia de una de estas maneras:

- desde Programas (F2): se selecciona y se pulsa RETURN, o se hace doble clic
- desde el Escritorio o el Commander: ARCHIVO > EJECUTAR
- desde otro programa, como Studio, que ejecuta lo que usted haga

Cada programa tiene su propio menú junto a los comunes VER y AYUDA. Termina desde su menú (normalmente BACK o STOP, CONTROL+Q) y vuelve a la vista desde la que se inició.

Varios programas a la vez

Los programas de V-OS son de dos clases:

- **Pequeños, con tabla de reubicación:** DEMO, MIXED y CLOCK, los ejemplos del editor web (PONG, CATCH, STARCAT, COUNTER), y lo que haga el SDK. Comparten 12K de memoria, y funcionan juntos tantos como quepan.
- **Programas que necesitan toda la memoria:** STUDIO y lo que hace Studio (CALC, MINES, FLAP, TICKER y los suyos). Sin ampliación de memoria funcionan solos. Si ya hay uno en marcha, aparece "CIERRE PRIMERO OTRO PROGRAMA".

Cuando se va a una vista (F1–F4) mientras un programa está en marcha:

- Un programa pequeño se queda en memoria. Se retoma desde Programas (RETURN sobre él) o con F6, y continúa exactamente donde estaba.
- Un programa que tiene toda la memoria termina, a menos que haya una ampliación de memoria.

F6 (VER > PROGRAMAS) muestra la lista de los programas en marcha. Se elige uno con las flechas y RETURN. Si no hay ninguno en marcha, aparece "NINGÚN PROGRAMA EN MARCHA".

Con una ampliación de memoria (p. ej. 512K, que V-OS encuentra por sí solo al arrancar), cada programa que necesita toda la memoria recibe su propio banco. Así, STUDIO, CALC, DEMO y PONG pueden funcionar todos juntos.

En segundo plano

Un programa que está detrás de otro, o detrás de una vista, normalmente **se congela** y continúa cuando vuelve al frente.

Algunos programas piden **seguir funcionando** también en segundo plano, p. ej. TICKER, que cuenta el tiempo. Mientras están ahí no dibujan ni suenan. Cuando vuelven al frente, también han contado el tiempo que estuvieron fuera.

En Studio esto se elige para su programa con STUDIO > IN BACKGROUND.

El accesorio de escritorio (ACCESORIO)

Un programa pequeño, como CLOCK, puede abrirse en una ventana **encima de cualquier cosa**: una vista o un programa.

1. En Programas, seleccione CLOCK y elija ARCHIVO > COMO ACCESORIO (^A).

2. A partir de entonces, **F5** lo abre y lo cierra, esté donde esté. ESC también lo cierra.

Mientras está abierto, recibe las teclas y los clics. Su ventana se puede arrastrar por su título. Cuando se cierra, se vuelve a dibujar lo que había debajo.

Los programas del disquete

Programa	Qué es
STUDIO	el editor de bloques: se hacen programas en el CPC (capítulo 5)
DEMO	el primer programa de V-OS: un cuadrado que se mueve solo. Las flechas lo dirigen y SPACE cambia su color
MIXED	la "pantalla mixta": una franja de 16 colores (modo 0) entre franjas de modo 1. Un clic muestra las coordenadas
CLOCK	el tiempo desde que arrancó V-OS. También funciona como accesorio de escritorio
CALC	una calculadora en una ventana con botones, hecha en Studio (también existe `CALC.VBS`)
MINES	un buscaminas de 8×6 con 8 minas, hecho en Studio. Un clic abre una casilla, y NEW empieza una partida nueva
BRICKS	romper ladrillos: una pala con ←/→, 32 ladrillos, 3 vidas, niveles con una pelota cada vez más rápida. En GAME OVER, SPACE para una partida nueva
VROUVES	tower defence, del juego de CPC LET'S GO FOR VROUVES: 7 pantallas, del Kafenio al cementerio, con 10 oleadas cada una. El disquete de V-OS tiene la pantalla 1. Todas están en su propio disquete, `vrouves.dsk`: en B:, y ARCHIVO > EJECUTAR en el Commander. Las flechas mueven el cursor, 1–6 elige un defensor, SPACE construye, S vende, N trae la oleada siguiente, P hace pausa. Escrito con el SDK (`myapps/vrouves`)
PONG	una pelota y una pala con las flechas, puntuación y vidas
CATCH	atrapar las estrellas que caen
STARCAT	un gato con disfraces que camina, una estrella, una melodía a partir de una lista
COUNTER	un contador con botones +1, −1 y 0, en modo 1
FLAP	un pájaro que bate las alas (disfraces), y una estrella que cambia de disfraz cada segundo
TICKER	cuenta segundos y sigue en segundo plano

- Los programas de bloques son archivos `.VBN`. Tienen un menú PROGRAM > STOP (CONTROL+Q).
- Los que tienen también un `.VBS` (CALC, MINES, FLAP, TICKER) se abren en Studio para verlos o cambiarlos.
- `VBRUN.SYS` no es un programa: es el "reproductor" que necesita Studio.

4. Idiomas

V-OS habla seis idiomas. El inglés viene incorporado. Los otros cinco son **paquetes de idioma**, archivos del disquete que traen tres cosas: todos los textos del sistema, el teclado y las letras (la fuente) del idioma.

Idioma	Archivo	Teclado
English	incorporado (`ENGLISH.MSG`)	el inglés del CPC
Ελληνικά	`GREEK.LNG`	griego, con acentos
Français	`FRENCH.LNG`	AZERTY, cifras con SHIFT, teclas muertas ^ y ¨
Español	`SPANISH.LNG`	Ñ en la tecla `:`, teclas muertas ´ ¨ ` ^, y ¡ ¿
Deutsch	`GERMAN.LNG`	QWERTZ, Ü Ö Ä ß, teclas muertas ´ y `
Italiano	`ITALIAN.LNG`	è é ò à ù ì en sus propias teclas

Cambiar de idioma

- **De forma permanente:** en Ajustes (F4), la línea IDIOMA recorre todos los idiomas. Cada uno se carga al momento desde A:, y toda la pantalla se vuelve a dibujar en él. Al guardar (^S) se mantiene también para el siguiente arranque.
- **Rápidamente:** CONTROL+L (VER > CAMBIAR DE IDIOMA) cambia entre el inglés y el idioma del paquete cargado.
- **Solo el teclado:** CONTROL+SHIFT cambia el teclado entre el inglés y el del paquete. La barra de menús muestra a la derecha cuál está activo (EN, EΛ, FR, ...).

Los símbolos `@ # [ ] { } \ |` no están en los teclados de los paquetes, porque el CPC no tiene AltGr. Escríbalos con el teclado inglés.

Los textos de los programas

Los programas tienen sus propios textos en **dos idiomas**: inglés y, de forma opcional, uno más (hoy, griego). Un programa aparece en su segundo idioma solo cuando el paquete está en ese idioma. Si no, aparece en inglés. Así, con el francés, Studio aparece en inglés.

Crear un idioma nuevo

Las fuentes de los idiomas están en `lang/<código>/`:

- `messages.txt`: los textos del sistema. La lista base la define `lang/en/messages.txt`, y lo que falte en un paquete aparece en inglés.
- `keymap.txt`: las teclas que difieren del teclado inglés.
- `pack.txt`: el código, el nombre, el juego de caracteres y la fuente.

`tools/mklng.py` los convierte en un `.LNG`, y el Makefile lo pone en el disquete. Los paquetes latinos comparten la fuente `lang/latin1/font.txt`.

5. Studio: programas con bloques

STUDIO es un editor de bloques al estilo de Scratch que funciona en el propio CPC.

- Se hacen sprites y scripts con bloques.
- El programa se ejecuta al momento con CONTROL+R.
- Se convierte en un programa normal de V-OS (`.VBN`) que también se inicia desde Programas (F2).

La pantalla

Studio usa una pantalla "dividida", con dos zonas:

- **Arriba, en modo 0 (16 colores):**
  - **a la izquierda, el escenario** a la mitad de tamaño
  - **a la derecha, el panel SPRITES:** los sprites con su color y su posición, el tamaño del seleccionado (W, H), el fondo y el modo del programa (p. ej. `STAGE 2 M1`), y el nombre del archivo
- **Abajo, en modo 1:**
  - **a la izquierda, la paleta** con los bloques de una categoría (`< EVENTS >`)
  - **a la derecha, el script**, una línea por bloque
  - **al final, la línea de estado**, con las teclas que funcionan donde se encuentra

TAB pasa del script a la paleta, luego al escenario, y vuelve al script.

Menús

STUDIO		EDIT
NEW	^N	DELETE LINE
OPEN (`.VBS`)	^O	+ SPRITE (hasta 6)
SAVE	^S	− SPRITE
SAVE AS		BACKGROUND (el color del escenario)
RUN	^R	SCREEN MODE (0 o 1)
IN BACKGROUND		
BACK	^Q	

- **RUN (^R):** guarda `NOMBRE.VBS`, escribe `NOMBRE.VBN` y lo ejecuta. El programa se detiene desde su propio menú (PROGRAM > STOP, ^Q), y entonces Studio vuelve a arrancar con el mismo programa abierto.
- **IN BACKGROUND (en segundo plano):** pregunta si el programa sigue funcionando cuando está detrás de otro o detrás de una vista, o se pausa, que es lo predeterminado.
- **SCREEN MODE (modo de pantalla):**
  - **Modo 0:** 16 colores, 192 píxeles de ancho.
  - **Modo 1:** 4 colores (tintas 0–3), 384 píxeles de ancho, y más texto.

La paleta

←/→ cambian la categoría, ↑/↓ eligen un bloque, y RETURN lo añade debajo de la línea del script. Un clic en un bloque de la paleta hace lo mismo.

Un bloque que abre (FOREVER, REPEAT, IF) entra junto con su END, y se borra junto con él. Un bloque con texto pide el texto al momento.

El script

Tecla	Qué hace
↑/↓	línea
←/→	hueco de la línea (los huecos son los valores dentro del bloque)
RETURN	sin hueco seleccionado: añade el bloque de la paleta. En un hueco de texto: cambia el texto. En un hueco de tecla: espera a que pulse la tecla
CLR	borra la línea
DEL	borra la última cifra de un número
ratón	clic en un hueco: lo selecciona. Segundo clic: cambia su clase

El programa es una lista de líneas:

- Cada `WHEN …` empieza un script, para un sprite o para el escenario. El script llega hasta el siguiente `WHEN`.
- La sangría sale sola de los bloques que abren y cierran.
- Las líneas antes del primer `WHEN` se ignoran.
- Lo que quede abierto se cierra al final del script.

Qué va en un hueco

La línea de estado siempre dice qué acepta el hueco.

Hueco de valor:

Tecla	Valor
0–9, −	número, un entero de −32768 a 32767
A–H	variable. La misma letra otra vez: A1, A2 … A15, y vuelta a A
X, Y	la posición del sprite del script
R y cifras	RANDOM: un número de 1 a n
K y una tecla	KEY: 1 mientras se mantiene pulsada, si no 0
T y una cifra	TOUCHING: 1 cuando el sprite toca el sprite n
Z	AT EDGE: 1 cuando el sprite está en el borde del escenario
P	BUTTON: el número del último botón pulsado
U	COSTUME: el número del disfraz del sprite (0: sin disfraz)

Otras clases de hueco:

- **Variable:** A–H, y la misma letra otra vez para A1…A15.
- **Comparación:** `<`, `=`, `>`. SPACE da NONE ("ninguna"), y entonces IF solo comprueba que el primer valor no sea 0.
- **Sprite** (en los bloques `WHEN`): 0 el escenario, 1–6 un sprite.
- **Tecla** (en `WHEN KEY`): RETURN y luego la tecla.
- **Texto:** RETURN, y se escriben hasta 14 caracteres.

- **Grupo de variables** (SAVE/LOAD VARS): A–H pasa al siguiente grupo de ocho (A-H, A1-H1 … A15-H15), y SPACE da ALL.

Las letras se leen como teclas, así que también funcionan con el teclado griego.

El escenario y los sprites

Tecla	Qué hace
flechas	mueven el sprite seleccionado
1–6	seleccionan un sprite
C	color siguiente
W, H	ancho, alto
K	qué disfraz lleva (ninguno, 1, 2, …)
E	dibujar su disfraz (uno nuevo, si no tiene)
clic en el escenario	pone el sprite ahí

Un sprite sin disfraz es un rectángulo de color.

Disfraces

E abre el editor de disfraces en lugar del escenario. Tiene una cuadrícula de 16×16 con las 16 tintas debajo.

Tecla	Qué hace
flechas	casilla
SPACE, RETURN	pinta la casilla
0–9, A–F	tinta
.	transparente
E, TAB	terminado
clic	en una casilla: pinta. En la fila de tintas: selecciona una tinta

Un programa tiene hasta 8 disfraces, compartidos por todos los sprites. El tamaño de un sprite con disfraz es 16×16. En el escenario, x se redondea a un byte entero (2 píxeles en modo 0, 4 en modo 1).

Todos los bloques

En la columna "Bloque", los huecos aparecen como en el script.

EVENTS

Bloque	Qué hace
WHEN START *sprite*	el script empieza junto con el programa
WHEN KEY *tecla sprite*	empieza cuando se pulsa la tecla
WHEN CLICKED *sprite*	empieza con un clic en el sprite (o en el escenario)
WHEN I RECEIVE *n sprite*	empieza cuando se envía el mensaje n
SEND *n*	envía el mensaje n (n también puede ser una variable)

> Un mensaje inicia **desde el principio** todos los scripts que lo esperan, aunque ya estuvieran en marcha. Por eso un script no debe enviarse un mensaje a sí mismo.

CONTROL

Bloque	Qué hace
FOREVER … END	repite para siempre
REPEAT *n* … END	n veces
IF *a comparación b* … [ELSE …] END	ejecuta las líneas solo si se cumple la comparación
WAIT *n*	espera n fotogramas (50 = un segundo)
WAIT UNTIL *a comparación b*	espera hasta que se cumpla la comparación
STOP	termina este script

MOTION

Bloque	Qué hace
GO TO *x y*	el sprite va a x, y
CHANGE X *n*, CHANGE Y *n*	se desplaza n
SPEED *vx vy*	fija su velocidad
MOVE	un paso a su velocidad
BOUNCE	en el borde del escenario, invierte la velocidad

LOOKS/SOUND

Bloque	Qué hace
COLOUR *n*	la tinta del rectángulo
COSTUME *n*	se pone el disfraz n (1–8. 0 no cambia nada)
NEXT COSTUME	el siguiente, y después del último otra vez el primero
SHOW, HIDE	
PRINT *valor* AT *x y*	escribe un número
NOTE *n* FOR *fotogramas*	toca una nota (número MIDI: 60 = do, 69 = la 440 Hz) y espera a que termine

VARIABLES

Bloque	Qué hace
SET *variable* TO *valor*	
CHANGE *variable* BY *valor*	suma
DECREASE *variable* BY *valor*	resta
MULTIPLY *variable* BY *valor*	
DIVIDE *variable* BY *valor*	división entera
REMAINDER *variable* BY *valor*	el resto de la división
SAVE VARS *grupo* TO *"nombre"*	escribe las variables en un archivo de texto (véase más abajo)
LOAD VARS *grupo* FROM *"nombre"*	las vuelve a leer

WINDOWS

Bloque	Qué hace
WINDOW *"título" ancho alto*	una ventana en el centro del escenario, como los diálogos de V-OS. Una ventana nueva ocupa el lugar de la anterior y de sus botones
BUTTON *"texto"* AT *x y*	un botón del sistema. Se pulsa con un clic o con TAB y RETURN. Los botones se numeran desde 0, en el orden en que aparecen
WHEN BUTTON *sprite*	empieza cuando se pulsa cualquier botón. El valor BUTTON (P) dice cuál

TEXT *"texto"* AT *x y*	escribe un texto
LABEL *botón* TO *"texto"*	cambia el texto de un botón

Con una ventana abierta, TEXT, BUTTON y PRINT cuentan desde la esquina superior izquierda de su interior. La ventana se arrastra por su título, junto con sus botones.

Variables

Hay 128 variables, compartidas por todos los scripts, llamadas A–H, A1–H1, …, A15–H15. Al principio todas valen 0, y contienen enteros de −32768 a 32767.

Guardar y recuperar (SAVE VARS, LOAD VARS)

- `SAVE VARS ALL TO "GAME"` escribe las 128 en el archivo `GAME.TXT`. El `.TXT` se añade cuando el nombre no tiene tipo.
- Con un grupo (`A-H`, `A1-H1`, …) escribe solo esas 8.
- El archivo es de texto, una línea por variable:

```
A=12 B=0 A1=-5
```

- `LOAD VARS ALL FROM "GAME"` cambia las variables que menciona el archivo. Con un grupo, cambia solo las que también están en el grupo. Las líneas que no entiende se ignoran, y si falta el archivo no cambia nada.
- Así, un archivo puede recuperar todas las variables o solo un grupo de ocho, p. ej. las posiciones guardadas de un juego.

Archivos

- **`NOMBRE.VBS`:** el programa tal como se edita. OPEN lo abre, y se leen hasta 5K.
- **`NOMBRE.VBN`:** el programa listo para ejecutarse. RUN lo escribe, y también se inicia desde Programas (F2). Es el reproductor (`VBRUN.SYS`) junto con el programa. El reproductor compila los bloques a código Z80 al arrancar, así que los programas van rápido.

Un `.VBS` también se puede hacer a partir de texto en el PC, con `tools/mkvbs.py`. La sintaxis se describe al principio del archivo, y hay ejemplos en `blocks/examples/*.vbt` (CALC, MINES, FLAP, TICKER, VARS).

Límites

Líneas (bloques)	240
Sprites	6
Scripts (`WHEN …`)	16
Variables	128
REPEAT en un programa	64 (a partir de ahí comparten un contador)
Bloques abiertos anidados unos dentro de otros	16
Textos	32, de hasta 14 caracteres cada uno
Disfraces	8, de 16×16
Botones	64

Código	al menos 4.000 bytes, incluso para el programa más grande. Lo que no cabe se ignora

Consejos

- **Scripts del escenario:** los bloques de movimiento y de apariencia no hacen nada, y X, Y, TOUCHING y AT EDGE dan 0.
- **TOUCHING:** solo se cumple cuando los dos sprites son visibles.
- **Sincronización:** los scripts funcionan "a la vez". Cada script cede su turno al final de cada bucle y en cada WAIT. Para que un script espere a otro, use una variable y `WAIT UNTIL` (véase MINES).
- **Ejemplos ya hechos:** CALC, MINES, FLAP y TICKER se abren en Studio (OPEN).

6. Bloques en el PC: el editor web y `vbc.py`

Además de Studio, que funciona en el CPC, existe también un editor de bloques para el PC. El lenguaje es el mismo, con algunos bloques más: listas, ruido, tamaño del sprite, texto de color.

El editor

Abra `blocks/editor.html` en un navegador. Está basado en Blockly y viene en griego o en inglés.

- **Categorías:** eventos, control, movimiento, apariencia, texto, operadores, sensores, variables, listas y ventanas.
- **Sprites:** rectángulos o disfraces. Se definen en una tabla, y el escenario los muestra con los colores del CPC. Se pueden arrastrar con el ratón.
- **Archivos:** abre y guarda archivos `.vbp`. Son JSON con todo el programa, así que también se pueden escribir a mano.

Compilar

```
python3 blocks/vbc.py pong.vbp --dsk build/vos.dsk
```

`vbc.py` escribe `build/NOMBRE.ASM`, con un comentario para cada bloque. Lo ensambla con RASM en `NOMBRE.VBN`, con una tabla de reubicación para que funcione junto a otros programas, y con `--dsk` lo pone en el disquete.

Opción	Qué hace
`-o build`	carpeta de salida
`--asm-only`	solo el `.ASM`
`--dsk build/vos.dsk`	también en el disquete

El formato `.vbp`

La descripción completa está al principio de `blocks/vbc.py`: eventos, órdenes, valores, disfraces, icono. En resumen:

```
{"name": "PONG", "mode": 0, "background": 0,
 "sprites": [{"name": "ball", "x": 90, "y": 100, "w": 6, "h": 6,
              "colour": 8, "vx": 2, "vy": 2, "visible": true}],
 "variables": {"score": 0},
 "scripts": [{"sprite": "ball", "when": {"event": "start"},
              "do": [{"block": "forever", "do": [{"block": "move"}, {"block": "bounce"}
```

- **Variables y listas:** tienen nombres, no letras. Las listas tienen una longitud máxima fija.
- **Icono:** `"icon"` da un icono para Programas (F2).
- **Un programa con muchas partes parecidas:** lo puede escribir un script de Python. Véase `tools/mkbricks.py`, que escribe `bricks.vbp` con 32 ladrillos.

Ejemplos

En `blocks/examples/`:

Archivo	Programa
`pong.vbp`	PONG

`catch.vbp`	CATCH
`starcat.vbp`	STARCAT: disfraces, una melodía a partir de una lista
`counter.vbp`	COUNTER: una ventana y botones
`bricks.vbp`	BRICKS (escrito por `tools/mkbricks.py`)

Los archivos `*.vbt` son programas de Studio en forma de texto, para `tools/mkvbs.py`. Véase el capítulo 5.

¿Studio o editor web?

	Studio	**editor web**
Dónde	en el CPC	en el PC
Archivo	`.VBS`	`.vbp`
Variables	128, A–H15	con nombres
Listas, ruido, tamaño del sprite	no	sí
SAVE/LOAD VARS	sí	no
El programa	necesita toda la memoria	pequeño, funciona junto a otros

7. El SDK de V-OS: programas en ensamblador

Con este SDK se escriben programas propios en ensamblador Z80. Funcionan dentro de V-OS como DEMO o CLOCK: aparecen en Programas (F2), en el Escritorio y en el Commander, y tienen su propio menú, botones, diálogos, sonido y archivos. También funcionan junto a otros programas.

Inicio rápido

```
make new-app NAME=hello
```

Esto crea la carpeta `myapps/hello/` a partir de `sdk/template/`. Contiene tres archivos:

- `hello.asm`: el programa. Las flechas mueven un recuadro, hay un botón BEEP, un contador de segundos y su propio menú (ABOUT, BACK) junto a VER y AYUDA.
- `texts.txt`: sus textos, en inglés y en griego.
- `icon.txt`: su icono, de 16×16, para Programas (F2).

```
make app NAME=hello
```

Esto compila `build/HELLO.BIN` y lo pone en el disquete `build/vos.dsk`.

```
make run
```

Esto abre el disquete en Caprice32. HELLO está en Programas (F2), en la última página.

- Un `make` a secas compila todos los programas de `myapps/` y los pone en el disquete, junto con V-OS.
- El nombre tiene de 1 a 8 letras minúsculas o cifras y empieza por una letra. No puede ser el nombre de un programa de V-OS (p. ej. demo).
- Se necesitan RASM, Python 3 e iDSK, como para todo V-OS.
- En Caprice32 el programa se ejecuta a mano, desde Programas (F2).

Un programa más grande:

- Ponga más archivos `.asm` en su carpeta (p. ej. `art.asm` con sprites) e inclúyalos con `include "art.asm"`. El build los encuentra y vuelve a compilar el programa cuando cambian.
- Lo que ponga en la subcarpeta `disc/` (p. ej. `disc/LEVEL1.DAT`) va tal cual al disquete, para `VOS_LOAD`.
- Con una subcarpeta `owndisc/`, el programa tiene además su propio disquete, `build/<nombre>.dsk`, con el `.BIN` y los archivos de `owndisc/`. Se pone en B: y se ejecuta desde el Commander (ARCHIVO > EJECUTAR). La unidad desde la que se inició, para cargar de allí sus archivos, está en `VOS_SELF`: `db unidad`, y luego su nombre (11 bytes).
- Un programa con tabla de reubicación (como los del SDK) puede ocupar hasta 12K. Para lo demás, cargue archivos en el banco de datos.

Cómo funciona un programa

Un programa es un `.BIN`, compilado para `VOS_APP_ADDR` (`#4000`). Empieza así:

```
        include "api/vos_api.inc"
        org VOS_APP_ADDR
```

```
        jp init                 ; +0
        jp event                ; +3  HL -> el evento
        jp redraw               ; +6
        db "VA", VOS_API_VERSION
        include "app_icon.asm"  ; +12 el icono (opcional)
```

- **`init`** fija el modo, la paleta, el menú y los botones, activa `VE_FRAME` si hace falta, y lo dibuja todo.
  - A = 0: un arranque nuevo.
  - A = 2: el programa vuelve de una vista o de otro programa. Su estado es tal como lo dejó; vuelve a fijar lo anterior y dibuja.
- **`event`** se llama para cada evento y vuelve al momento. Un programa nunca espera por su cuenta (sin bucle de espera). Para movimiento o tiempo, `VOS_TICKS` con A = 1, y entonces llega un `VE_FRAME` 50 veces por segundo.
- **`redraw`** lo vuelve a dibujar todo, p. ej. cuando cambia el idioma o se cierra el accesorio de escritorio.
- **Salida:** `VOS_EXIT`, que vuelve a la vista que lo inició.

El orden dentro de `event`

```
event:  ld a,(hl)
        cp VE_FRAME : jp z,frame        ; los fotogramas
        push hl
        call VOS_UI_EVENT               ; menús, botones, CONTROL+letra
        pop hl
        or a : jr nz,command            ; A = la orden
        call VOS_MENU_OPEN : ret nz     ; menú abierto: las teclas son suyas
        ...                             ; sus propias teclas y clics
command: ...
        jp VOS_COMMAND                  ; lo que no es suyo (VER, AYUDA)
```

Mientras hay un menú abierto, no dibuje. Si no, dibujará encima de él. En `VE_FRAME`, actualice el estado, pero dibuje solo si `VOS_MENU_OPEN` da Z. Los diálogos (`VOS_DIALOG`, `VOS_INPUT`, `VOS_FILE_PICK`) tienen su propio bucle, así que no llega ningún evento mientras están abiertos.

Los eventos

`HL` → 8 bytes: `db type, code, char, mods; dw x, y`.

type	qué	code / char / x, y
`VE_KEY_DOWN` 1, `VE_KEY_REPEAT` 3, `VE_KEY_UP` 2	tecla	code = número de tecla (`VK_*`, fila del teclado·8 + bit), char = el carácter en el idioma del paquete
`VE_PTR_DOWN` 5, `VE_PTR_UP` 6, `VE_PTR_MOVE` 4	puntero	x, y en píxeles del modo
`VE_FRAME` 8	fotograma (con `VOS_TICKS`)	—
`VE_LANG` 7	cambió el idioma del teclado	—

Tres clases de programa

- **Con tabla de reubicación (lo predeterminado del SDK):** `make app` lo compila dos veces, en

`#4000` y en `#4100`. `tools/mkreloc.py` añade la tabla a partir de la diferencia.

    - Funciona junto a otros programas, en los 12K desde `#5000`, y continúa donde se detuvo (F6 muestra los programas en marcha).
    - Sus datos deben estar dentro de sus propios bytes (etiquetas, `ds`), nunca en direcciones fijas.
    - Si `mkreloc.py` da un error, en algún sitio se usa una dirección de una manera que no sobrevive a la reubicación. P. ej. `ld a,label & 255`, o solo el byte bajo de una dirección.

- **Sin tabla:** ocupa todo el banco (hasta 16K, código y datos), pero solo funciona solo, a menos que haya una ampliación de memoria. Necesita su propia regla en el Makefile (véase `STUDIO.BIN`).
- **Accesorio de escritorio:** hasta 4K, con tabla. Se abre con F5 encima de cualquier cosa, e `init` recibe A = 1. Véanse `apps/clock/clock.asm` y la descripción en `src/api/vos_api.inc`.

Reglas

- **Registros:** las llamadas alteran todos los registros excepto IX. Los argumentos van en A, BC, DE, HL (IX para `VOS_RECT` y `VOS_BLIT`). Los resultados vuelven en A o HL. Si hay fallo, se activa el acarreo (carry) y A = número de error, que `VOS_ERROR` muestra.
- **Memoria:** no ponga otro banco en `#4000`, porque el programa desaparecerá bajo sus pies. Para 16K más está el banco de datos (`VOS_BANK_GET`, `VOS_BANK_PUT`, `VOS_DATA`). Se comparte con las vistas y otros programas, y conserva sus datos solo mientras el programa está en marcha.
- **Coordenadas:** x en píxeles del modo (modo 0: 0–191, modo 1: 0–383), y de 0 a 258. La barra de menús ocupa y 0–11.
- **RASM:** las etiquetas no distinguen mayúsculas de minúsculas (`BOX` y `box` son lo mismo). El hexadecimal se escribe con `#` y no con `&`.

Los formatos de datos

- **Barra de menús** (`VOS_SET_BAR`): `db count; dw menu, ...`. Los menús comunes VER y AYUDA vienen de `VOS_MENU_VIEW` y `VOS_MENU_HELP`.
- **Menú:** `db title, count` y para cada opción `db text, command, letter, 0`.

    - text: `VOS_SEPARATOR` para una línea
    - command: 1.., lo que devuelve `VOS_UI_EVENT`
    - letter: atajo CONTROL+letra, o 0

- **Botones** (`VOS_SET_WIDGETS`, luego `VOS_DRAW_WIDGETS`): `db count`, y para cada botón `dw x, y; db ancho en caracteres, text, command, 0`.
- **Diálogo** (`VOS_DIALOG` → A = botón, 0 el primero, ESC da el último): `db title, text, número de botones (1-3), button, ...`. Para los botones, `VM_OK`, `VM_YES` y `VM_NO`.
- **Textos** (`VOS_TEXTS`): los genera `tools/mktexts.py` a partir de `texts.txt`, con líneas `NAME | English | Greek`.

    - Da `T_NAME` (`VT_FIRST` = `#C0` en adelante) y la tabla `texts:`. Hasta 64 textos.
    - Un `|` sin espacios alrededor empieza una línea nueva en los diálogos.
    - Los textos van donde se pide un "texto" (menús, botones, diálogos). Con `VOS_MSG` se

obtiene su dirección para `VOS_PRINT`.

- **Paleta** (`VOS_PALETTE`): 17 colores del firmware (0–26): 16 tintas y el borde. En modo 1 cuentan las tintas 0–3.
- **Rectángulo** (`VOS_RECT`, IX →): `dw x, y, w, h; db pen`.
- **Icono:** 16 líneas de 16 caracteres en `icon.txt`. Cada carácter es una tinta en hexadecimal (modo 0, paleta de Programas) o `.` para transparente.
- **Nombres de archivo** (`VOS_LOAD`, `VOS_SAVE`, `VOS_DELETE`, `VOS_RUN`): 11 bytes, `"NAME EXT"`. Unidad 0 = A:, 1 = B:.

La API (versión 11)

Llamada	Entrada → salida	Qué hace
`VOS_EXIT`	—	fin, vuelta a la vista que lo inició
`VOS_TICKS`	A = 1 / 0	`VE_FRAME` 50 veces por segundo, o ninguno
`VOS_FRAMES`	→ HL (y A bits 16–23)	fotogramas desde el arranque
`VOS_RANDOM`	→ A	0–255
`VOS_COMMAND`	A = orden	las órdenes comunes de VER y AYUDA
`VOS_ERROR`	A = error	diálogo de error
`VOS_MODE`	A = 0 / 1	modo 0 (192×259, 16 tintas) o 1 (384×259, 4 tintas)
`VOS_PALETTE`	HL → 17 colores	
`VOS_CLS`	A = tinta	borra la pantalla
`VOS_PLOT`	HL = x, DE = y, A = tinta	un píxel
`VOS_RECT`	IX → rectángulo	rectángulo relleno
`VOS_COLOURS`	A = tinta del texto, C = fondo	para `VOS_PRINT` y `VOS_CHAR`
`VOS_PRINT`	HL = x, DE = y, BC → texto, 0	
`VOS_CHAR`	HL = x, DE = y, A = carácter	
`VOS_MSG`	A = texto → HL	la dirección de un texto (`T_*` o uno del sistema)
`VOS_SET_BAR`, `VOS_DRAW_BAR`	HL → barra de menús	
`VOS_UI_EVENT`	(el evento actual) → A	orden de menú, botón o atajo, 0 si no hay
`VOS_DIALOG`	HL → diálogo → A	
`VOS_INPUT`	A = título, HL → texto, C = máximo → carry si se cancela	introducción de texto (necesita C + 2 bytes)
`VOS_SET_WIDGETS`, `VOS_DRAW_WIDGETS`	HL → botones	
`VOS_MENU_VIEW`, `VOS_MENU_HELP`	→ HL	los menús comunes
`VOS_CATALOG`	A = unidad → A = archivos, HL → lista	16 bytes cada uno: 11 el nombre, en +12 el tamaño en K
`VOS_FILE_PICK`	A = título → A = archivo, carry si se cancela	diálogo selector, después de `VOS_CATALOG`

VOS_LOAD	A = unidad, HL → nombre, DE = dónde → BC = longitud	
VOS_SAVE	A = unidad, HL → nombre, DE = desde, BC = longitud	archivo binario, que sustituye al anterior
VOS_DELETE	A = unidad, HL → nombre	
VOS_TEXTS	HL → tabla de textos	los textos del programa
VOS_MENU_OPEN	→ NZ si hay un menú abierto	
VOS_KEY_HELD	A = tecla → NZ mientras está pulsada	para juegos
VOS_BLIT	IX → `dw x, y; db bytes, lines; dw source; db source stride`	imagen: pares (máscara, píxeles)
VOS_SOUND	A = canal 0–2, HL = periodo (bit 15: ruido), C = volumen 0–15, B = fotogramas	una nota; 440 Hz es el periodo 142
VOS_SPLIT	HL → `dw y1, y2; db 8 colores` (0 = la desactiva)	pantalla mixta: modo 0 entre y1 e y2
VOS_RUN	A = unidad, HL → nombre	ejecuta otro programa, y después este vuelve a arrancar
VOS_KEEP	→ HL → 16 bytes	memoria que se mantiene entre programas
VOS_BANK_GET, VOS_BANK_PUT	HL = desde, DE = hasta, BC = bytes	copia desde y hacia el banco de datos
VOS_BACKGROUND	A = 1 / 0	sigue funcionando o se congela cuando está detrás de otro
VOS_DRAG	HL = x, DE = y, BC = w, A = h → carry: nuevos HL, DE	arrastrar una ventana por su título
VOS_FIT	igual → HL, DE	mete una ventana dentro de la pantalla

Todos los detalles, p. ej. sobre el segundo plano, el accesorio y `VOS_SPLIT`, están en `src/api/vos_api.inc`. Como ejemplos, véanse `apps/demo/demo.asm` (modo 0, movimiento), `apps/clock/clock.asm` (accesorio) y `apps/mixed/mixed.asm` (pantalla mixta). Un juego completo con sprites, una pantalla leída de un archivo y sonido es `myapps/vrouves/`.

Pruebas

`make test-sdk` compila la plantilla como TEMPLATE.BIN y la abre desde Programas (F2) en Caprice32, con pulsaciones de teclas definidas por `src/kernel/test_script.asm` (script 15). Hace capturas de pantalla en `build/shots`: su pantalla, el recuadro después de moverse, su menú, el diálogo ABOUT y la vuelta a Programas.

Para su propia prueba automática, copie el script 15 y el objetivo `test-sdk` del Makefile.

8. Desarrollo de V-OS

Para quienes trabajan en el propio V-OS. Los programas *para* V-OS se describen en el SDK.

Qué se necesita

- RASM, el ensamblador
- iDSK, para las imágenes de disquete
- Python 3, para las herramientas. `make docs` necesita además `markdown` y `pymupdf`.
- GNU make 4.3 o posterior
- para las pruebas, Caprice32 (snap `caprice32`)

Compilación

Orden	Qué hace
`make`	`build/vos.dsk`: el núcleo, las vistas, los idiomas, todos los programas y `myapps/*`
`make run`	abre el disquete en Caprice32 (`RUN"VOS"`)
`make shot`	arranca sin ventana y hace una captura de pantalla
`make docs`	el manual en PDF: `docs/V-OS.pdf` (griego) y `docs/V-OS-en.pdf`, `-fr`, `-es`, `-de`, `-it`
`make windows`	`build/V-OS-windows.zip`: `V-OS.exe` para Windows, con Caprice32 (CPC 6128, 576K), `vos.dsk` en A: y `vrouves.dsk` en B:. V-OS arranca solo. Hace falta WSL y el `cap32-win64.zip` de Caprice32 (`CAP32_ZIP=…`)
`make new-app NAME=x`, `make app NAME=x`	su propio programa (SDK)
`make clean`	borra `build/`

Estructura del repositorio

Carpeta	Contenido
`src/vos.asm`	la raíz de la compilación: cargador y núcleo, escritura en el disquete
`src/boot/`	el cargador que AMSDOS ejecuta en `#4000`, detección de la ampliación de RAM
`src/kernel/`	vectores, interrupciones, sistema de archivos, vistas, API (`api.asm`), ajustes, extensión del núcleo (`ext.asm`), scripts de prueba (`test_script.asm`)
`src/drivers/`	Gate Array, CRTC, teclado, ratón AMX, FDC, AY
`src/gfx/`	gráficos overscan, texto, pantalla mixta
`src/ui/`	menús, botones, diálogos, introducción de texto, selector de archivos
`src/shell/`	las vistas como overlays: Escritorio, Programas, Commander, Ajustes
`src/api/vos_api.inc`	la API para los programas
`lang/`	los idiomas (textos, teclados, fuentes)
`apps/`	los programas en ensamblador: DEMO, MIXED, STUDIO, CLOCK, y sus iconos
`blocks/`	el editor web, `vbc.py`, `runtime.asm`, el reproductor de Studio (`player.asm`), ejemplos
`sdk/`	el SDK: guía y plantilla de programa

`tools/`	herramientas: fuentes, idiomas, textos, iconos, reubicación, `.VBS` a partir de texto, capturas de pantalla, PDF
`docs/`	este manual en griego (`manual/`) y en inglés, francés, español, alemán e italiano (`manual/en/`, `fr/`, `es/`, `de/`, `it/`), los PDF, los requisitos (`REQUIREMENTS.md`)

La memoria, en resumen

Zona	Qué contiene
`#0000-#3DFF`	el núcleo. En la página cero, después de los vectores de reinicio, la tabla de la API en `#0108`
hasta `#4000`	la pila del núcleo
`#4000-#7FFF`	la "ventana" de bancos
`#8000-#FFFF`	la pantalla overscan de 32K. En sus huecos, búferes del núcleo

Los bancos que entran en la ventana:

Banco	Qué contiene
4	búfer de archivos y de carga, y el banco de datos del programa en marcha
5	la vista en `#4000-#4FFF`, y desde `#5000` los programas con tabla de reubicación (o un programa sin tabla en todo el banco)
6	lo que tapan los menús y los diálogos, y desde `#7000` el accesorio de escritorio
7	el paquete de idioma, las listas del Commander, los textos ingleses, y desde `#7000` la extensión del núcleo

Con una ampliación de memoria, sus bancos acogen programas. Los detalles, dirección por dirección, están en `docs/REQUIREMENTS.md` (§4.2 y en cada sección "Υλοποίηση M..." (implementación)).

Convenciones

- **RASM:** el hexadecimal se escribe `#C000` y no `&C000`, porque RASM lee `&` como AND. Las etiquetas no distinguen mayúsculas de minúsculas.
- **Comentarios:** en inglés, breves, como en el código existente. Cada archivo empieza con un bloque que dice qué hace.
- **Documentación:** en griego. `README.md` y `docs/REQUIREMENTS.md` tienen una sección para cada hito. El manual se escribe en griego y se traduce a los otros cinco idiomas de V-OS.
- **Textos:** los textos del sistema están en `lang/*/messages.txt`, y los textos de los programas en `texts.txt` (para `tools/mktexts.py`).

Pruebas

Las pruebas ejecutan V-OS en Caprice32 sin ventana y hacen capturas de pantalla en `build/shots/<retardo>/`.

Cómo funcionan:

- Caprice32 sin ventana no puede pulsar flechas, COPY ni teclas f. Por eso las pulsaciones las hace una compilación especial: un `-DTEST_SCRIPT=n` pone el script n en `src/kernel/test_script.asm`, una tabla de teclas por fotograma.
- Cada objetivo `make test-…` compila un disquete con su script y hace una captura en cada retardo de `…_SHOTS`.
- El retardo se mide en pulsaciones de "Z": unos 2,5–3,5 fotogramas cada una. La correspondencia no es fija, así que un momento se captura con dos o tres retardos a su

alrededor.

- Una vez que V-OS escribe en el disquete, Caprice32 no se cierra solo, y la prueba termina por tiempo agotado. Esto es lo esperado en test-vars, test-costume y otras.
- Las capturas se miran a mano, p. ej. combinadas en una sola imagen con `ffmpeg … hstack/vstack`.

Objetivo	Qué prueba
`test-input`	teclas, Studio: bloques, archivos, ejecución
`test-calc`	CALC con clics, luego `CALC.VBS` en Studio
`test-costume`	disfraz en Studio, ejecución, reapertura
`test-acc`	el reloj como accesorio de escritorio
`test-multi`, `test-switch`, `test-expand`, `test-behind`	varios programas, F6, ampliación de memoria, segundo plano
`test-drag`	arrastrar ventanas
`test-langs`, `test-lang`	los idiomas, guardar el idioma
`test-mines`, `test-bricks`	los juegos
`test-vars`	las 128 variables, SAVE/LOAD VARS
`test-sdk`	la plantilla del SDK
`test-vrouves`	VROUVES desde su disquete en B:, por el Commander: construir, la oleada 1, el dinero
`test-amsdos`, `test-reboot`, `test-system`	archivos de V-OS en el BASIC, reinicio, disquete SYSTEM

Script nuevo:

1. Añada un `IF TEST_SCRIPT == n` al principio de la cadena en `test_script.asm`, con `ELSE` antes del siguiente y un `ENDIF` más al final.
2. Añada un objetivo al Makefile, como `test-sdk`.

Cada tecla se escribe como `dw desde, hasta : db fila, bit` del teclado. El número de tecla es fila·8 + bit, p. ej. RETURN = 18 = fila 2, bit 2.

Sin emulador: el código de un programa también funciona en un Z80 dentro de Python (el paquete `z80`). Se carga el `.BIN` en `#4000` y se hace que las llamadas a la API vuelvan al momento (o hagan lo necesario, p. ej. archivos en un diccionario). Luego se llama a init y event y se lee la memoria. Es mucho más rápido que las capturas para errores en números, archivos y saltos. Las direcciones de las etiquetas salen de `rasm … -s -sl` (un archivo `.sym`).

En cada hito

1. Código, y una prueba en Caprice32: nueva o existente, con capturas de pantalla.
2. Las pruebas a las que afecta el cambio.
3. Una sección en `README.md` y una sección "Υλοποίηση Μ..." (implementación) en `docs/REQUIREMENTS.md`.
4. **Actualizar el manual en los seis idiomas** (`docs/manual/` en griego, `docs/manual/<código>/` para en, fr, es, de, it, y `sdk/README.md`, `sdk/README.<código>.md` si cambia la API), y luego `make docs` para todos los PDF.
5. Commit y push.